

Session 2: Clustering, Outlier Detection, Train/Test and Generalization

موضوع کلی

تمرکز این جلسه روی مفاهیم زیر است:

- Unsupervised Learning
- Clustering
- Distance
- Feature Scaling
- Outlier Detection
- Train/Test Split
- Evaluation
- Generalization
- Underfitting
- Overfitting

راهنمای سطح سوال‌ها

● Core: سوال‌های اصلی که همه باید انجام دهند

● Stretch: سوال‌های کمی چالشی‌تر

🎁 Bonus: سوال اختیاری برای گروه‌هایی که زمان اضافه دارند

● بخش 1 – ساخت Dataset مشتریان

داده‌ی زیر اطلاعات ۱۲ مشتری یک فروشگاه آنلاین را نشان می‌دهد.

ستون‌ها به ترتیب عبارت‌اند از:

```
# age, monthly_visits, avg_order_value, total_purchases
```

```
customers = np.array([
```

```
    [22, 2, 15, 1],
```

```
    [25, 4, 20, 2],
```

```
    [47, 10, 120, 12],
```

```
    [52, 12, 150, 15],
```

```
    [35, 6, 60, 6],
```

```
    [38, 7, 70, 7],
```

```
    [19, 1, 10, 1],
```

```
    [60, 8, 130, 10],
```

```
    [41, 9, 90, 9],
```

```
    [28, 5, 35, 4],
```

```
    [33, 6, 55, 5],
```

```
    [55, 11, 160, 14]
```

```
])
```

به سوال‌های زیر پاسخ دهید:

1. مقدارهای `ndim`, `shape`, `size`, `dtype` را برای آرایه‌ی `customers` چاپ کنید.

2. چند `sample` در این `dataset` وجود دارد؟

3. چند `feature` برای هر `customer` داریم؟

4. در یک `comment` کوتاه بنویسید:

○ هر row نشان دهنده‌ی چیست؟

○ هر column نشان دهنده‌ی چیست؟

5. آیا این dataset در حال حاضر label دارد یا نه؟

6. چون label نداریم، این مسئله بیشتر به کدام نوع learning نزدیک است؟

Supervised Learning

Unsupervised Learning

7. دلیل پاسخ خود را در یک markdown cell بنویسید.

بخش 2 – بررسی آماری Feature ها ●

قبل از ساختن model یا انجام clustering ، باید dataset را بهتر بشناسیم.

1. برای هر feature مقدارهای زیر را حساب کنید:

min

max

mean

std

2. خروجی‌ها را طوری چاپ کنید که مشخص باشد هر مقدار مربوط به کدام feature است.

Feature names:

```
feature_names = np.array([
```

```
    "age",
```

```
    "monthly_visits",
```

```
    "avg_order_value",
```

```
    "total_purchases"
```

1)

3. feature ای که بیشترین mean را دارد پیدا کنید.
4. feature ای که بیشترین std را دارد پیدا کنید.
5. در یک markdown cell توضیح دهید چرا feature ای مثل avg_order_value ممکن است روی distance تاثیر زیادی بگذارد.

● بخش 3 Feature Scaling –

چون feature ها scale های متفاوتی دارند، قبل از محاسبه ی distance باید scaling انجام دهیم.
مثلا:

- age حدودا بین ۱۹ تا ۶۰ است.
- monthly_visits بین ۱ تا ۱۲ است.
- avg_order_value بین ۱۰ تا ۱۶۰ است.
- total_purchases بین ۱ تا ۱۵ است.

اگر scaling انجام نشود، feature هایی با عددهای بزرگتر ممکن است تاثیر بیشتری روی distance بگذارند.

1. برای هر feature مقدار min و max را حساب کنید.
2. با استفاده از Min-Max Scaling آرایه ای به نام customers_scaled بسازید.

فرمول:

$$\text{customers_scaled} = (\text{customers} - \text{customers_min}) / (\text{customers_max} - \text{customers_min})$$

3. مقدار customers_scaled را چاپ کنید.
4. بررسی کنید که همه ی مقدارهای customers_scaled بین ۰ و ۱ هستند.

راهنمایی:

```
np.min(customers_scaled)
```

```
np.max(customers_scaled)
```

5. میانگین feature ها را قبل و بعد از scaling چاپ کنید.

6. در یک comment توضیح دهید چرا scaling برای clustering و distance مهم است.

● بخش 4 – چهار Reference Point برای Clustering

در این بخش می‌خواهیم هر customer را به نزدیک‌ترین گروه نسبت دهیم.

فرض کنید چهار نوع customer داریم:

```
# age, monthly_visits, avg_order_value, total_purchases
```

```
ref_points = np.array([
    [23, 2, 15, 1], # low activity
    [32, 5, 45, 4], # normal
    [42, 8, 85, 8], # loyal
    [55, 12, 150, 14] # VIP
])
```

```
ref_names = np.array([
    "low activity",
    "normal",
    "loyal",
```

"VIP"

l)

1. آرایه‌ی ref_points را با همان customers_min و customers_max مربوط به dataset اصلی scale کنید.

2. فاصله‌ی هر customer تا هر reference point را بدون استفاده از loop حساب کنید.

راهنمایی:

خروجی distance باید shape زیر را داشته باشد:

(12, 4)

یعنی:

- customer_{۱۲}

- reference point_۴

3. برای هر customer ، نزدیک‌ترین reference point را با np.argmin پیدا کنید.

4. برای هر customer نام cluster مربوط به او را چاپ کنید.

5. خروجی باید شبیه این باشد:

Customer 1 -> low activity

Customer 2 -> normal

Customer 3 -> VIP

...

6. در یک markdown cell توضیح دهید چرا این کار شبیه یک روش ساده‌ی Clustering است.

بخش 5 – تحلیل Cluster ها ●

حالا که برای هر customer یک cluster داریم، می‌خواهیم هر cluster را تحلیل کنیم.

1. تعداد اعضای هر cluster را حساب کنید.
2. برای هر cluster ، میانگین feature ها را حساب کنید.
3. خروجی را به شکل زیر چاپ کنید:

Cluster: VIP

Count: ...

Mean age: ...

Mean monthly visits: ...

Mean avg order value: ...

Mean total purchases: ...

4. بررسی کنید کدام cluster بیشترین avg_order_value را دارد.
5. بررسی کنید کدام cluster بیشترین monthly_visits را دارد.
6. در یک markdown cell بنویسید:

- آیا cluster ها منطقی به نظر می‌رسند؟
- customer های VIP چه ویژگی‌هایی دارند؟
- customer های low activity چه ویژگی‌هایی دارند؟

● بخش 6 Label – ساختن با Threshold

در این بخش می‌خواهیم از روی یک rule ساده برای customer ها label بسازیم.
قانون:

اگر مقدار total_purchases بزرگ‌تر یا مساوی ۸ باشد، customer را "valuable" در نظر می‌گیریم.
در غیر این صورت، customer را "regular" در نظر می‌گیریم.

```
# if total_purchases >= 8 => valuable
```

else => regular

1. با استفاده از np.where آرایه‌ای به نام customer_label بسازید.
2. مقدار customer_label را چاپ کنید.
3. تعداد customer های "valuable" و "regular" را حساب کنید.
4. میانگین avg_order_value را برای customer های valuable و regular جداگانه حساب کنید.
5. میانگین monthly_visits را برای customer های valuable و regular جداگانه حساب کنید.
6. در یک markdown cell توضیح دهید:
 - این label با کمک rule ساخته شد یا model؟
 - اگر بخواهیم یک model بسازیم که valuable/regular را پیش‌بینی کند، مسئله از نوع Classification است یا Regression؟

● بخش 7 Outlier Simulation –

- در این بخش می‌خواهیم یک customer غیرعادی یا Outlier بسازیم.
- Customer زیر مقدار avg_order_value بسیار بزرگی دارد، اما تعداد خرید او کم است:
- ```
outlier_customer = np.array([[30, 1, 500, 1]])
```
1. این customer را به dataset اصلی اضافه کنید و آرایه‌ای به نام customers\_with\_outlier بسازید.
- راهنمایی:

np.concatenate(...)

2. مقدار customers\_with\_outlier.shape را چاپ کنید.
3. داده‌ی جدید را با Min-Max Scaling scale کنید.



4. مرکز داده‌ها را با mean حساب کنید.

```
center = np.mean(customers_with_outlier_scaled, axis=0)
```

5. فاصله‌ی هر customer تا center را با np.linalg.norm حساب کنید.

6. customer ای که بیشترین فاصله را از center دارد پیدا کنید.

راهنمایی:

```
np.argmax(...)
```

7. بررسی کنید آیا customer پیدا شده همان outlier است یا نه.

8. در یک markdown cell توضیح دهید:

- Outlier یعنی چه؟
- چرا این customer می‌تواند outlier باشد؟
- آیا برای پیدا کردن outlier در این تمرین label داشتیم؟
- پس این کار به Supervised Learning نزدیک‌تر است یا Unsupervised Learning؟

---

## بخش 8 Distance Matrix – کامل

تا اینجا فاصله‌ی هر customer را با چند reference point یا center حساب کردیم.

حالا می‌خواهیم فاصله‌ی همه‌ی customer ها با همه‌ی customer های دیگر را حساب کنیم.

1. با استفاده از customers\_scaled و np.newaxis یک ماتریس distance بسازید.

راهنمایی:

```
diff = customers_scaled[:, np.newaxis, :] - customers_scaled[np.newaxis, :, :]
```

2. با استفاده از np.linalg.norm روی axis مناسب، ماتریس distance را بسازید.

3. خروجی باید shape زیر را داشته باشد:

(12, 12)

4. مقدار قطر اصلی ماتریس را چاپ کنید.

راهنمایی:

```
np.diag(distance_matrix)
```

5. در یک comment توضیح دهید چرا مقدارهای قطر اصلی باید صفر باشند.

6. برای هر customer نزدیک‌ترین customer دیگر را پیدا کنید.

نکته مهم:

خود customer نباید به عنوان نزدیک‌ترین انتخاب شود.

راهنمایی:

می‌توانید فاصله‌ی هر customer با خودش را برابر یک عدد بزرگ مثل np.inf قرار دهید.

```
np.fill_diagonal(distance_matrix, np.inf)
```

7. خروجی را به شکل زیر چاپ کنید:

```
Customer 1 nearest neighbor is Customer ...
```

```
Customer 2 nearest neighbor is Customer ...
```

```
...
```

---

## ● بخش 9 Mini Classification with Nearest Reference

در این بخش می‌خواهیم یک customer جدید را دسته‌بندی کنیم.

Customer جدید:

```
new_customer = np.array([40, 9, 100, 9])
```

Featureها به ترتیب:

```
age, monthly_visits, avg_order_value, total_purchases
```

1. این customer را با همان customers\_min و customers\_max مربوط به dataset اصلی scale کنید.

2. فاصله‌ی customer جدید را با چهار ref\_points\_scaled حساب کنید.

3. نزدیک‌ترین reference point را پیدا کنید.

4. نام cluster مربوط به customer جدید را چاپ کنید.

5. با استفاده از threshold بخش 6 مشخص کنید این customer "valuable" است یا "regular".

6. خروجی نهایی باید شبیه این باشد:

Nearest cluster: loyal

Business label: valuable

7. در یک markdown cell توضیح دهید:

- cluster label چگونه به دست آمد؟
- business label چگونه به دست آمد؟
- تفاوت این دو نوع label چیست؟

---

## بخش 10 Train/Test Split – مفهومی

در Machine Learning ، هدف فقط خوب بودن روی داده‌های training نیست.

مدل باید روی داده‌های جدید هم عملکرد خوبی داشته باشد.  
به این توانایی Generalization می‌گوییم.

فرض کنید label واقعی train و test به شکل زیر است:

```
y_train = np.array(["pass", "fail", "pass", "pass", "fail", "pass"])
```

```
y_test = np.array(["pass", "fail", "fail", "pass"])
```

سه model مختلف داریم.

#### Model A

```
model_A_train_pred = np.array(["pass", "fail", "pass", "pass", "fail", "pass"])
```

```
model_A_test_pred = np.array(["fail", "pass", "pass", "fail"])
```

#### Model B

```
model_B_train_pred = np.array(["pass", "pass", "pass", "pass", "pass", "pass"])
```

```
model_B_test_pred = np.array(["pass", "pass", "pass", "pass"])
```

#### Model C

```
model_C_train_pred = np.array(["pass", "fail", "pass", "fail", "fail", "pass"])
```

```
model_C_test_pred = np.array(["pass", "fail", "fail", "pass"])
```

سوال‌ها:

1. برای هر model مقدار Train Accuracy را حساب کنید.

2. برای هر model مقدار Test Accuracy را حساب کنید.

فرمول: Accuracy:

```
accuracy = number_of_correct_predictions / total_number_of_predictions
```

3. نتیجه را به شکل زیر چاپ کنید:

Model A - Train Accuracy: ... Test Accuracy: ...

Model B - Train Accuracy: ... Test Accuracy: ...

Model C - Train Accuracy: ... Test Accuracy: ...

4. کدام model روی train بهترین عملکرد را دارد؟

5. کدام model روی test بهترین عملکرد را دارد؟

6. در یک markdown cell توضیح دهید چرا فقط Train Accuracy کافی نیست.

---

## بخش 11 – تشخیص Overfitting و Underfitting ●

با توجه به نتایج بخش قبل، به سوال‌های زیر پاسخ دهید.

1. اگر یک model روی train accuracy خیلی بالا داشته باشد، اما روی test accuracy خیلی پایین باشد، احتمالاً چه مشکلی دارد؟

Overfitting

Underfitting

Good Generalization

2. اگر یک model هم روی train و هم روی test عملکرد ضعیفی داشته باشد، احتمالاً چه مشکلی دارد؟

Overfitting

Underfitting

Good Generalization

3. اگر یک model روی train و test عملکرد قابل قبول و نزدیک به هم داشته باشد، چه وضعیتی دارد؟

Overfitting

Underfitting

Good Generalization

4. برای Model A, Model B, Model C مشخص کنید هرکدام بیشتر به کدام حالت نزدیک هستند:

Overfitting

Underfitting

## Good Generalization

5. دلیل خود را برای هر model در یک markdown cell بنویسید.

---

### بخش 12 Regression Error – و Generalization ●

در این بخش به جای Classification ، یک مسئلهی Regression داریم.

مقدارهای واقعی و prediction های سه model روی train و test داده شده‌اند.

```
y_train_reg = np.array([50, 75, 38, 92, 65, 84])
```

```
y_test_reg = np.array([55, 80, 60, 90])
```

#### Model A

```
model_A_train_pred = np.array([50, 75, 38, 92, 65, 84])
```

```
model_A_test_pred = np.array([30, 100, 40, 120])
```

#### Model B

```
model_B_train_pred = np.array([60, 60, 60, 60, 60, 60])
```

```
model_B_test_pred = np.array([60, 60, 60, 60])
```

#### Model C

```
model_C_train_pred = np.array([52, 73, 40, 88, 66, 82])
```

```
model_C_test_pred = np.array([57, 78, 63, 88])
```

سوال‌ها:

1. برای هر model مقدار Train MAE را حساب کنید.

2. برای هر model مقدار Test MAE را حساب کنید.

فرمول: Absolute Error

```
absolute_error_i = abs(predicted_i - actual_i)
```

فرمول: MAE

$MAE = \text{mean}(\text{abs}(\text{predicted} - \text{actual}))$

3. نتیجه را به شکل زیر چاپ کنید:

Model A - Train MAE: ... Test MAE: ...

Model B - Train MAE: ... Test MAE: ...

Model C - Train MAE: ... Test MAE: ...

4. کدام model احتمالا overfitting دارد؟

5. کدام model احتمالا underfitting دارد؟

6. کدام model بهترین generalization را دارد؟

7. در یک markdown cell دلیل خود را توضیح دهید.

---

### ● بخش 13 Underfitting – با Feature کم

فرض کنید می‌خواهیم final\_score دانشجویان را پیش‌بینی کنیم.

Dataset:

```
study_hours, attendance, previous_score
```

```
X = np.array([
```

```
 [2, 60, 45],
```

```
 [5, 80, 70],
```

```
 [1, 40, 35],
```

```
 [8, 90, 88],
```

```
 [4, 75, 60],
```

```
[7, 85, 80]
```

```
)
```

```
y = np.array([50, 75, 38, 92, 65, 84])
```

دو prediction ساده داریم.

### Model Simple

این model فقط از study\_hours استفاده می‌کند:

```
prediction_simple = X[:, 0] * 10
```

### Model Better

این model از سه feature استفاده می‌کند:

```
prediction_better = 0.4 * X[:, 0] * 10 + 0.3 * X[:, 1] + 0.3 * X[:, 2]
```

سوال‌ها:

1. برای هر model مقدار absolute\_errors را حساب کنید.
  2. برای هر model مقدار MAE را حساب کنید.
  3. کدام model بهتر است؟
  4. در یک markdown cell توضیح دهید:
- چرا Model Simple ممکن است underfitting داشته باشد؟
  - چرا استفاده از feature های بیشتر می‌تواند به prediction کمک کند؟
  - آیا همیشه feature بیشتر بهتر است؟ چرا؟

---

## ● بخش 14 Overfitting – با Train/Test Error

فرض کنید سه model روی train و test خطاهای زیر را دارند:



```
models = np.array(["Model A", "Model B", "Model C"])
```

```
train_error = np.array([2, 25, 8])
```

```
test_error = np.array([40, 30, 10])
```

1. اختلاف  $\text{test\_error} - \text{train\_error}$  را برای هر model حساب کنید.
  2. مدلی را پیدا کنید که  $\text{train error}$  خیلی کم ولی  $\text{test error}$  خیلی زیاد دارد.
  3. این model را به عنوان model دارای Overfitting چاپ کنید.
  4. مدلی را پیدا کنید که هم  $\text{train error}$  و هم  $\text{test error}$  زیاد دارد.
  5. این model را به عنوان model دارای Underfitting چاپ کنید.
  6. مدلی را پیدا کنید که  $\text{test error}$  کمتری دارد.
  7. این model را به عنوان model با Best Generalization چاپ کنید.
  8. در یک markdown cell توضیح دهید چرا  $\text{test error}$  برای بررسی generalization مهمتر از  $\text{train error}$  است.
- 

## ● بخش 15 – جمع‌بندی Session 2

در یک markdown cell به سوال‌های زیر پاسخ دهید:

1. Unsupervised Learning یعنی چه؟
2. چرا Clustering معمولا Unsupervised است؟
3. در این تمرین cluster label چگونه ساخته شد؟
4. Outlier یعنی چه؟
5. چرا ممکن است outlier روی model تاثیر منفی بگذارد؟
6. Train/Test Split برای چیست؟

7. Generalization یعنی چه؟

8. Overfitting یعنی چه؟

9. Underfitting یعنی چه؟

10. اگر train accuracy بالا و test accuracy پایین باشد، مشکل چیست؟

11. اگر train accuracy و test accuracy هر دو پایین باشند، مشکل چیست؟

12. هدف اصلی Machine Learning چیست: خوب بودن فقط روی train data یا عملکرد خوب روی داده‌های جدید؟

---

### Bonus 1 – VIP Leaderboard

با استفاده از reference point مربوط به VIP:

```
vip_ref = ref_points_scaled[3]
```

1. فاصله‌ی همه‌ی customer ها تا VIP reference point را حساب کنید.

2. customer ها را از نزدیک‌ترین تا دورترین نسبت به VIP مرتب کنید.

راهنمایی:

```
np.argsort(...)
```

3. یک leaderboard چاپ کنید:

```
Rank 1: Customer ... Distance ...
```

```
Rank 2: Customer ... Distance ...
```

```
Rank 3: Customer ... Distance ...
```

```
...
```

4. در یک markdown cell بنویسید top 3 customer چه ویژگی‌هایی دارند.

---

## 🎁 Bonus 2 – Confusion Matrix ساده

برای داده‌ی زیر:

```
true = np.array(["pass", "fail", "pass", "pass", "fail", "fail"])
```

```
pred = np.array(["pass", "pass", "pass", "fail", "fail", "fail"])
```

برای کلاس "pass" مقدارهای زیر را بدون استفاده از sklearn حساب کنید:

1. True Positive

2. True Negative

3. False Positive

4. False Negative

5. Accuracy

راهنمایی:

```
true == "pass"
```

```
pred == "pass"
```

---